

# Midterm Study Guide

UNC Chapel Hill - MADS  
DATA780: Machine Learning  
Spring 2026

## Table of Contents

|                                                      |    |
|------------------------------------------------------|----|
| Foundations of statistical learning.....             | 6  |
| Quick definitions: .....                             | 6  |
| • Statistical learning.....                          | 6  |
| • Supervised learning .....                          | 6  |
| • Unsupervised learning .....                        | 6  |
| • Regression .....                                   | 6  |
| Core NumPy / computational foundations .....         | 7  |
| Quick definition: .....                              | 7  |
| • Broadcasting.....                                  | 7  |
| Linear algebra background for machine learning ..... | 8  |
| • Know definitions and interpretations of: .....     | 8  |
| • Understand matrix operations and concepts: .....   | 8  |
| • Quick definitions: .....                           | 9  |
| ○ Scalar .....                                       | 9  |
| ○ Vector .....                                       | 9  |
| ○ Basis .....                                        | 9  |
| ○ Span.....                                          | 9  |
| ○ Column.....                                        | 9  |
| ○ Null space.....                                    | 9  |
| ○ Orthonormal .....                                  | 9  |
| • Core equations:.....                               | 9  |
| ○ Dot product.....                                   | 9  |
| ○ Euclidean norm .....                               | 10 |
| ○ Euclidean distance .....                           | 10 |
| ○ Rank-nullity.....                                  | 10 |
| ○ Eigenvector relation .....                         | 10 |
| Linear regression .....                              | 11 |
| Core equations:.....                                 | 11 |

|                                                 |    |
|-------------------------------------------------|----|
| ○ Scalar form .....                             | 11 |
| ○ Vector form .....                             | 11 |
| ○ SSE / RSS objective .....                     | 12 |
| ○ Matrix least squares solution .....           | 12 |
| ○ Ridge-regularized objective .....             | 12 |
| Helpful interpretations: .....                  | 12 |
| ○ Design matrix $\Phi$ .....                    | 12 |
| ○ Least squares .....                           | 12 |
| ○ Probabilistic model .....                     | 12 |
| Optimization concepts .....                     | 13 |
| Core equations: .....                           | 13 |
| ○ Gradient descent update .....                 | 13 |
| ○ SGD update .....                              | 13 |
| Key idea: .....                                 | 13 |
| ○ Closed-form solutions .....                   | 13 |
| Logistic regression and classification .....    | 14 |
| Core equations: .....                           | 14 |
| ○ Logit score .....                             | 14 |
| ○ Sigmoid .....                                 | 14 |
| ○ Binary probability model .....                | 14 |
| ○ Sigmoid derivative .....                      | 14 |
| ○ Negative log-likelihood / cross-entropy ..... | 15 |
| ○ Softmax for class $k$ .....                   | 15 |
| Important note: .....                           | 15 |
| ○ Logistic regression .....                     | 15 |
| Discriminative vs generative models .....       | 16 |
| Core equation: .....                            | 16 |
| ○ Bayes rule .....                              | 16 |
| Decision rule: .....                            | 16 |

|                                                              |    |
|--------------------------------------------------------------|----|
| ○ Bayes classifier .....                                     | 16 |
| Gaussian Bayes / generative classification .....             | 17 |
| Core density:.....                                           | 17 |
| Key interpretation: .....                                    | 17 |
| ○ If classes have different covariance matrices .....        | 17 |
| Conditional independence and Naive Bayes .....               | 18 |
| Core definitions and equations:.....                         | 18 |
| • Conditional independence .....                             | 18 |
| • Naive Bayes assumption .....                               | 18 |
| • Posterior up to proportionality: .....                     | 18 |
| Neural networks: big picture and core building blocks .....  | 19 |
| Core equations:.....                                         | 19 |
| ○ One hidden layer network .....                             | 19 |
| ○ For binary classification.....                             | 19 |
| Key idea: .....                                              | 19 |
| ○ Without a nonlinear activation .....                       | 19 |
| Neural networks: extended vocabulary and architectures ..... | 20 |
| Know terms such as:.....                                     | 20 |
| Understand convolutional neural networks (CNNs):.....        | 20 |
| Understand recurrent neural networks (RNNs):.....            | 20 |
| Quick definitions: .....                                     | 21 |
| ○ Logits .....                                               | 21 |
| ○ Tensor .....                                               | 21 |
| ○ Attention .....                                            | 21 |
| ○ CNNs .....                                                 | 21 |
| ○ RNNs .....                                                 | 21 |
| Validation, model assessment, and model selection .....      | 22 |
| Core concepts:.....                                          | 22 |
| ○ Model assessment .....                                     | 22 |

|                                                              |    |
|--------------------------------------------------------------|----|
| ○ Model selection .....                                      | 22 |
| Important relationship: .....                                | 23 |
| ○ In squared-error .....                                     | 23 |
| Cross-validation reminder:.....                              | 23 |
| ○ LOOCV .....                                                | 23 |
| ○ k-fold CV .....                                            | 23 |
| Tree-based methods .....                                     | 24 |
| Core equations:.....                                         | 25 |
| ○ Regression tree node prediction .....                      | 25 |
| ○ RSS for a split.....                                       | 25 |
| ○ Gini index .....                                           | 25 |
| ○ Cross-entropy / deviance .....                             | 25 |
| ○ Cost-complexity criterion .....                            | 25 |
| Key idea: .....                                              | 25 |
| ○ Larger trees.....                                          | 25 |
| Ensemble tree methods .....                                  | 26 |
| Quick definitions: .....                                     | 26 |
| ○ Bagging .....                                              | 26 |
| ○ Random forest .....                                        | 26 |
| ○ Boosting .....                                             | 26 |
| Core aggregation rules: .....                                | 26 |
| ○ Regression bagging .....                                   | 26 |
| Conceptual comparisons that are especially exam-worthy ..... | 28 |
| Formulas / relationships worth knowing cold .....            | 29 |
| • Linear regression.....                                     | 29 |
| • Least squares .....                                        | 29 |
| • Normal equation .....                                      | 29 |
| • Sigmoid .....                                              | 29 |
| • Logistic regression loss.....                              | 29 |

|                                                                       |    |
|-----------------------------------------------------------------------|----|
| • Gradient descent .....                                              | 29 |
| • Bayes rule.....                                                     | 29 |
| • Naive Bayes .....                                                   | 29 |
| • Bias-variance-noise .....                                           | 29 |
| • Tree impurity .....                                                 | 29 |
| Practical interpretation / short-answer themes likely to appear ..... | 30 |

# Foundations of statistical learning

- Define statistical learning and explain its goals: prediction, pattern discovery, and inference from data.
- Distinguish **supervised learning** from **unsupervised learning**.
- Recognize common task types:
  - regression for quantitative outputs
  - classification for categorical outputs
  - clustering and dimensionality reduction for unlabeled data
- Explain why combining multiple predictors can improve prediction over using a single variable.
- Interpret examples such as wage prediction, stock-market direction, and gene-expression clustering.

## Quick definitions:

- **Statistical learning** = using data to estimate relationships between predictors  $X$  and an outcome  $Y$ .
- **Supervised learning** = learn from labeled pairs  $(x_i, y_i)$ .
- **Unsupervised learning** = learn structure from unlabeled data  $x_i$  only.
- **Regression** predicts a numeric response; **classification** predicts a class label; **clustering** groups similar observations without labels.

- Core idea: we often want a rule  $f$  such that  $Y \approx f(X)$ , where  $X$  may be a single predictor or a vector of many predictors.

## Core NumPy / computational foundations

- Explain **broadcasting** and why it matters for efficient array computation.
- State the broadcasting compatibility rules:
  - compare shapes from the right
  - dimensions are compatible if equal or one equals 1
  - otherwise NumPy raises an error
- Understand scalar expansion and shape matching without unnecessary copying.

### Quick definition:

- **Broadcasting** lets NumPy perform elementwise operations on arrays of different shapes when dimensions are compatible.
  - Useful shape examples:
    - $(n, p) + (p, ) \rightarrow (n, p)$
    - $(n, 1) + (1, p) \rightarrow (n, p)$
    - scalar  $c$  can broadcast to any array shape



# Linear algebra background for machine learning

- Know definitions and interpretations of:
  - scalar
  - vector
  - vector space
  - subspace
  - basis
  - span
  - column space / column rank
  - null space / nullity
  - orthonormality
- Understand matrix operations and concepts:
  - matrix multiplication
  - transpose
  - inner product / dot product
  - outer product
  - Hadamard product
  - determinant
  - trace

- norm and distance
- Euclidean distance
- Be able to explain **rank-nullity**.
- Know the role of **eigenvalues**, **eigenvectors**, and **eigendecomposition**.
- Understand why these ideas matter for ML representations and optimization.
- Quick definitions:
  - **Scalar**: a single number.
  - **Vector**: an ordered list of numbers, often viewed as a point or direction in space.
  - **Basis**: a linearly independent set that spans a vector space.
  - **Span**: all linear combinations of a set of vectors.
  - **Column space**: all linear combinations of a matrix's columns.
  - **Null space**: all vectors  $x$  such that  $Ax = 0$ .
  - **Orthonormal**: vectors are mutually orthogonal and each has norm 1.
- Core equations:
  - **Dot product**:  $x^T y = \sum_{j=1}^p x_j y_j$

- **Euclidean norm:**  $\|x\|_2 = \sqrt{\sum_{j=1}^p x_j^2}$
- **Euclidean distance:**  $d(x, y) = \|x - y\|_2$
- **Rank-nullity:**  $\text{rank}(A) + \text{nullity}(A) = p$  for an  $m \times p$  matrix  $A$
- **Eigenvector relation:**  $Av = \lambda v$

## Linear regression

- Define linear regression as a supervised learning model for predicting real-valued outputs.
- Write the linear model with weights and intercept.
- Explain the sum of squared errors loss.
- Derive or interpret the least-squares solution.
- Understand the design matrix and the closed-form solution:
- $w^* = (\Phi^T \Phi)^{-1} \Phi^T t$
- Explain the geometric interpretation of least squares as projection onto the column space.
- Understand basis functions and how they allow nonlinear structure to be modeled in transformed space.
- Explain the probabilistic interpretation:
- Gaussian noise assumption
- least squares as maximum likelihood estimation
- Interpret the predictive distribution and prediction uncertainty.
- Explain overfitting and the purpose of regularization.

### Core equations:

- **Scalar form:**  $\hat{y} = w_0 + w_1 x_1 + \dots + w_p x_p$
- **Vector form:**  $\hat{y} = w^T x + b$

- **SSE / RSS objective:**  $\text{RSS}(w) = \sum_{i=1}^n (t_i - w^T \phi(x_i))^2$
- **Matrix least squares solution:**  $w^* = (\Phi^T \Phi)^{-1} \Phi^T t$
- **Ridge-regularized objective:**  $\sum_{i=1}^n (t_i - w^T \phi(x_i))^2 + \lambda \|w\|_2^2$

### Helpful interpretations:

- **Design matrix  $\Phi$**  has one row per observation and one column per feature or basis function.
- **Least squares** chooses the projection of  $t$  onto the column space of  $\Phi$ .
- **Probabilistic model:**  $t_i = w^T \phi(x_i) + \varepsilon_i$  with  $\varepsilon_i \sim \mathcal{N}(0, \sigma^2)$ .

## Optimization concepts

- Understand why optimization appears in model fitting.
- Distinguish:
  - analytic / closed-form solutions
  - iterative optimization
- Explain **gradient descent**:
  - initialization
  - gradient updates
  - learning rate / step size
  - convergence intuition
- Explain **stochastic gradient descent** and mini-batch reasoning.
- Know when iterative methods are needed instead of closed-form solutions.

### Core equations:

- **Gradient descent update**:  $w^{(k+1)} = w^{(k)} - \eta \nabla L(w^{(k)})$
- **SGD update**:  $w \leftarrow w - \eta \nabla \ell_i(w)$  using one observation (or a mini-batch) at a time

### Key idea:

- **Closed-form solutions** are exact algebraic formulas; iterative methods repeatedly improve parameters when no simple formula exists or when the data are too large for direct matrix methods.

## Logistic regression and classification

- Define logistic regression as a supervised classification method.
- Understand binary classification with the **sigmoid** / logistic function.
- Explain how logistic regression models  $P(Y = 1 | X = x)$ .
- Understand **likelihood**, **log-likelihood**, and **negative log-likelihood**.
- Know that logistic regression is often trained by minimizing cross-entropy / logistic loss.
- Explain why logistic regression generally does **not** have a simple closed-form analytic solution.
- Connect logistic regression training to gradient-based optimization.
- Understand the derivative identity for the sigmoid.
- Explain issues with **linearly separable data** and why **regularization** is needed.
- Recognize one-vs-all / one-vs-rest classification and **softmax** multiclass classification.

### Core equations:

- **Logit score:**  $z = w^T x + b$
- **Sigmoid:**  $\sigma(z) = \frac{1}{1+e^{-z}}$
- **Binary probability model:**  $P(Y = 1 | X = x) = \sigma(w^T x + b)$
- **Sigmoid derivative:**  $\sigma'(z) = \sigma(z)(1 - \sigma(z))$

- Negative log-likelihood / cross-entropy:

$$L(w, b) = - \sum_{i=1}^n [y_i \log p_i + (1 - y_i) \log(1 - p_i)], p_i = \sigma(w^T x_i + b)$$

- Softmax for class  $k$ :  $P(Y = k \mid X = x) = \frac{e^{z_k}}{\sum_j e^{z_j}}$

Important note:

- Logistic regression usually has no closed-form minimizer for the cross-entropy loss, so it is fit numerically.



## Discriminative vs generative models

- Distinguish **discriminative** models from **generative** models.
- Understand:
  - discriminative models target  $P(Y | X)$  directly
  - generative models work through joint or class-conditional structure such as  $P(X, Y)$  or  $P(X | Y)P(Y)$
- Use **Bayes rule** to connect class priors, class-conditionals, and posterior classification.
- Explain the **Bayes classifier** as the optimal classifier under the assumed model.

### Core equation:

- **Bayes rule:**  $P(Y = k | X = x) = \frac{P(X = x | Y = k)P(Y = k)}{P(X = x)}$

### Decision rule:

- **Bayes classifier** predicts the class with the largest posterior probability:

$$\hat{y}(x) = \arg \max_k P(Y = k | X = x)$$

## Gaussian Bayes / generative classification

- Understand how Gaussian class-conditionals are used in classification.
- Explain class priors and class-conditional densities.
- Know that each class may have its own mean and covariance.
- Understand why decision boundaries can be quadratic in Gaussian Bayes classifiers.
- Recognize the parameter-estimation burden for generative models.

### Core density:

- If class  $k$  is Gaussian, then

$$p(x | Y = k) = \frac{1}{\left(2\pi\right)^{\frac{p}{2}} |\Sigma_k|^{\frac{1}{2}}} \exp\left(-\frac{1}{2}(x - \mu_k)^T \Sigma_k^{-1} (x - \mu_k)\right)$$

### Key interpretation:

- If classes have different covariance matrices  $\Sigma_k$ , the discriminant function generally becomes quadratic in  $x$ .

# Conditional independence and Naive Bayes

- Define **conditional independence**.
- Explain the Naive Bayes assumption that features are independent given the class.
- Show how this simplifies the class-conditional probability into a product over features.
- Understand why Naive Bayes needs fewer parameters than a full generative model.
- Be prepared to discuss tradeoffs:
  - lower variance / less data demand
  - stronger assumptions that may be violated in practice

## Core definitions and equations:

- **Conditional independence** means that given  $Y$ , knowing one feature gives no extra information about another feature.
- **Naive Bayes assumption:**

$$P(X_1, \dots, X_p \mid Y = k) = \prod_{j=1}^p P(X_j \mid Y = k)$$

- **Posterior up to proportionality:**

$$P(Y = k \mid X = x) \propto P(Y = k) \prod_{j=1}^p P(X_j = x_j \mid Y = k)$$

## Neural networks: big picture and core building blocks

- Explain how modern neural networks relate to earlier models like logistic regression.
- Understand neural nets as learning **feature maps** and then applying linear or logistic prediction on top.
- Explain why **nonlinearities** are essential.
- Understand **layers**, **units**, **weights**, **biases**, and **hierarchical feature learning**.
- Explain function composition and why stacking purely linear maps is not enough.
- Interpret layer computations in matrix form.
- Understand neural networks as nonlinear, hierarchical function approximators.

### Core equations:

- One hidden layer network:  $h = g(W_1x + b_1)$ , then  $\hat{y} = W_2h + b_2$
- For binary classification, often  $\hat{p} = \sigma(W_2h + b_2)$

### Key idea:

- Without a nonlinear activation  $g$ , multiple layers collapse to one linear transformation, so depth would not buy you anything.

# Neural networks: extended vocabulary and architectures

Know terms such as:

- logits
- softmax
- tensors
- attention
- spatio-temporal data

Understand convolutional neural networks (CNNs):

- grid-structured inputs
- kernels / filters
- feature maps
- sparse interactions
- parameter sharing
- translation equivariance / shift handling
- pooling and its role

Understand recurrent neural networks (RNNs):

- sequence modeling
- hidden state / memory
- parameter sharing across time
- unfolding through time

- teacher forcing
- outputs at each step vs one output after the full sequence
- why order matters for sequential data

### Quick definitions:

- **Logits** are raw scores before sigmoid or softmax is applied.
- **Tensor** is a multidimensional array.
- **Attention** is a mechanism that lets a model weight which parts of the input are most relevant.
- **CNNs** exploit local spatial structure through shared filters.
- **RNNs** reuse the same parameters across time and update a hidden state sequentially.

# Validation, model assessment, and model selection

- Distinguish **training error** from **test / future prediction error**.
- Understand the goals of:
  - **model assessment**
  - **model selection**
- Explain the **bias-variance-noise** decomposition conceptually.
- Understand how model complexity affects bias and variance.
- Know the major validation methods:
  - test-set validation
  - leave-one-out cross-validation (LOOCV)
  - k-fold cross-validation
- Compare their pros and cons:
  - variance
  - computational cost
  - data efficiency
- Be able to explain why cross-validation is widely used.

## Core concepts:

- **Model assessment** asks: how well does the chosen model generalize?
- **Model selection** asks: which model complexity or tuning parameter should we choose?

### Important relationship:

- In squared-error settings, a common conceptual decomposition is

$$E \left[ \left( Y - \hat{f}(X) \right)^2 \right] = \text{Bias} \left( \hat{f}(X) \right)^2 + \text{Var} \left( \hat{f}(X) \right) + \sigma^2$$

where  $\sigma^2$  is irreducible noise.

### Cross-validation reminder:

- LOOCV uses  $n$  folds of size 1.
- $k$ -fold CV splits data into  $k$  parts, trains on  $k - 1$ , validates on the held-out fold, and averages the results.



## Tree-based methods

- Understand decision trees for both **regression** and **classification**.
- Define:
  - predictor space / feature space
  - regions / boxes
  - splits
  - recursive binary splitting
  - terminal nodes / leaves
  - internal nodes
- Explain the top-down greedy tree-building process.
- For regression trees, understand use of **RSS** and mean response within a region.
- For classification trees, understand:
  - classification error rate
  - class proportions
  - node purity
  - **Gini index**
  - **cross-entropy / deviance**
- Explain **overfitting** in trees and the role of **pruning**.
- Understand **cost-complexity pruning** and the tuning parameter  $\alpha$ .
- Explain how **cross-validation** is used to choose tree complexity.

## Core equations:

- Regression tree node prediction: average response in the region,  $\hat{y}_R = \frac{1}{|R|} \sum_{i: x_i \in R} y_i$
- RSS for a split:  $\sum_{m=1}^M \sum_{i: x_i \in R_m} (y_i - \hat{y}_{R_m})^2$
- Gini index:  $G = 1 - \sum_{k=1}^K \hat{p}_k^2$
- Cross-entropy / deviance:  $D = - \sum_{k=1}^K \hat{p}_k \log \hat{p}_k$
- Cost-complexity criterion:  $\sum_{m=1}^{|T|} \sum_{i: x_i \in R_m} (y_i - \hat{y}_{R_m})^2 + \alpha |T|$

## Key idea:

- Larger trees usually reduce training error but may increase test error, so pruning is used to control complexity.

## Ensemble tree methods

- Understand **bagging** as bootstrap aggregating.
- Explain bootstrap samples and why averaging many high-variance trees can reduce variance.
- Distinguish bagging behavior for:
  - regression trees (averaging predictions)
  - classification trees (majority vote)
- Understand **out-of-bag (OOB) error** as an internal estimate of test error.
- Recognize the tradeoff between predictive power and interpretability in ensembles.
- Expect **random forests** and **boosting** to be fair game conceptually, since they are explicitly named in the Week 9 overview.

### Quick definitions:

- **Bagging** = fit many trees on bootstrap samples, then aggregate predictions.
- **Random forest** = bagging plus random feature subsampling at each split.
- **Boosting** = sequentially fit weak learners so later ones focus on earlier errors.

### Core aggregation rules:

- **Regression bagging**:  $\hat{f}_{\text{bag}}(x) = \frac{1}{B} \sum_{b=1}^B \hat{f}^{(b)}(x)$

- Classification bagging: predict by majority vote across trees.

## Conceptual comparisons that are especially exam-worthy

- supervised vs unsupervised learning
- regression vs classification
- linear vs nonlinear models
- analytic solution vs gradient-based optimization
- discriminative vs generative modeling
- Gaussian Bayes vs Naive Bayes
- logistic regression vs neural networks
- CNNs vs RNNs
- training error vs test error
- test-set validation vs LOOCV vs k-fold CV
- single decision trees vs bagging / random forests / boosting

## Formulas / relationships worth knowing cold

- Linear regression:  $\hat{y} = w^T x + b$
- Least squares:  $\sum_{i=1}^n (y_i - \hat{y}_i)^2$
- Normal equation:  $w^* = (X^T X)^{-1} X^T y$
- Sigmoid:  $\sigma(z) = \frac{1}{1+e^{-z}}$
- Logistic regression loss:

$$-\sum_{i=1}^n [y_i \log p_i + (1 - y_i) \log(1 - p_i)]$$

- Gradient descent:  $w \leftarrow w - \eta \nabla L(w)$
- Bayes rule:  $P(Y | X) = \frac{P(X|Y)P(Y)}{P(X)}$
- Naive Bayes:  $P(X | Y) = \prod_{j=1}^p P(X_j | Y)$
- Bias-variance-noise:  $\mathbb{E}[(Y - \hat{f}(X))^2] = \text{Bias}^2 + \text{Var} + \sigma^2$
- Tree impurity:
  - RSS for regression trees
  - $G = 1 - \sum_k \hat{p}_k^2$  for Gini
  - $D = -\sum_k \hat{p}_k \log \hat{p}_k$  for cross-entropy

## Practical interpretation / short-answer themes likely to appear

- when a method is appropriate
- what assumptions a method makes
- what kind of output it predicts
- how model complexity affects generalization
- how to evaluate predictive performance honestly
- how regularization helps prevent overfitting
- why neural networks need nonlinear activations
- why Naive Bayes can still work even when its independence assumption is imperfect
- why ensemble methods often outperform a single tree